Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Program: Software Engineering

Operating Systems

Laboratory Work No. 8

The Same Calls, Inside Your Own OS

Track B - xv6

Elaborated by: st. gr. FAF-241, Ermicev Sofia

Verified by: Patricia Reițman

Chișinău - 2026

Contents

1 Objective **2**

2 Environment and Preparation **2**

3 Part 1 - run inside xv6 **2**
3.1 Program implementation 2
3.2 Execution with echo 3
3.3 Execution with ls 3
3.4 Failed exec 3
3.5 Observe answers 4

4 Part 2 - pingpong **4**
4.1 Program implementation 4
4.2 Pipe directions 5
4.3 Execution 5
4.4 Observe answers 5

5 Conclusion **6**

1. Objective

The purpose of this laboratory work was to repeat the process-management operations used in Linux in the xv6 operating system and to implement bidirectional communication between two processes using pipes. The work focused on `fork()`, `exec()`, `wait()`, `pipe()`, `read()`, and `write()`.

2. Environment and Preparation

The laboratory was performed in the existing xv6 RISC-V source tree:

```
cd ~/xv6-riscv
make qemu
```

The system built and booted successfully. New user programs were added to the `UPROGS` list in the xv6 `Makefile`.

The relevant entries were:

```
$U/_run\
$U/_pingpong\
```

3. Part 1 - run inside xv6

3.1. Program implementation

The first program repeats the same three-step model used in Linux: create a child process, replace it with another program, and wait for it to finish.

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("parent: my pid is %d\n", getpid());

  int pid = fork();
  if(pid < 0){
    printf("fork failed\n");
    exit(1);
  }

  if(pid == 0){
    printf("child: my pid is %d\n", getpid());
    char *args[] = { "echo", "hello", "from", "the", "child", 0 };
    exec("echo", args);
    printf("exec failed\n");
    exit(1);
  }
```

```
  int status;
  wait(&status);
  printf("parent: child %d ended with status %d\n", pid, status);
  exit(0);
}
```

3.2. Execution with echo

The first test used `echo` as the program executed by the child.

```
$ run
parent: my pid is 3
child: my pid is 4
hello from the child
parent: child 4 ended with status 0
```

The parent process created a child with `fork()`. The child was then replaced by `echo` using `exec()`. After the child terminated, the parent continued after `wait()` and printed the exit status.

3.3. Execution with ls

The program was then changed to use:

```
char *args[] = { "ls", 0 };
exec("ls", args);
```

The child successfully became `ls` and printed the xv6 root directory. The end of the run was:

```
parent: child 4 ended with status 0
```

This confirmed that the child process kept its process identity but changed the program that it was executing.

3.4. Failed exec

Finally, the code intentionally attempted to execute a program that did not exist:

```
exec("nosuchprog", args);
```

The result was:

```
$ run
parent: my pid is 3
child: my pid is 4
exec failed
parent: child 4 ended with status 1
```

The message `exec failed` was printed because `exec()` returned instead of replacing the process. The next statement, `exit(1)`, set the child's exit status to 1.

3.5. Observe answers

1. Calls in xv6 and Linux.

In xv6 the program uses:

- `fork()`
- `exec()`
- `wait()`

In Linux Lab 7 the corresponding calls were `fork()`, `execlp()`, and `waitpid()`. The overall model is the same: create a child, replace the child with another program, and wait for the child to terminate.

2. Who prints "hello from the child"?

The line is printed by the `echo` program. After a successful `exec("echo", args)`, the original child code is replaced by `echo`.

3. Why status 1 in xv6 and 127 in Linux?

In the xv6 program the failure path ends with `exit(1)`, therefore the parent receives status 1. In the Linux Lab 7 version the failure path used `exit(127)`, therefore the parent received 127.

4. Part 2 - pingpong

4.1. Program implementation

The second program uses two pipes so that the parent and child can send a byte in both directions.

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  int p2c[2], c2p[2];
  char b = 'x';

  pipe(p2c);
  pipe(c2p);

  if(fork() == 0){
    close(p2c[1]);
    close(c2p[0]);

    read(p2c[0], &b, 1);
    printf("%d: received ping\n", getpid());
    write(c2p[1], &b, 1);

    close(p2c[0]);
    close(c2p[1]);
```

```
    exit(0);
  }

  close(p2c[0]);
  close(c2p[1]);

  write(p2c[1], &b, 1);
  read(c2p[0], &b, 1);
  printf("%d: received pong\n", getpid());

  close(p2c[1]);
  close(c2p[0]);
  wait(0);
  exit(0);
}
```

4.2. Pipe directions

Two independent pipes were used:

```
p2c: parent -> child
c2p: child -> parent
```

For each pipe, index 0 is the read end and index 1 is the write end. The child closes the ends that it does not use, and the parent does the same.

4.3. Execution

The program produced:

```
$ pingpong
4: received ping
3: received pong
```

The child printed the ping message first. It could only do so after `read(p2c[0], &b, 1)` received the byte sent by the parent. Then the child wrote the byte back through the second pipe, allowing the parent to continue.

4.4. Observe answers

1. Why are two pipes needed?

A pipe is used as a one-direction communication channel in this program. One pipe carries data from parent to child, while the second carries data from child back to parent. This creates bidirectional communication without mixing the two directions.

2. Which call makes the parent wait for the reply?

The blocking call is:

```
read(c2p[0], &b, 1);
```

The parent cannot continue past this call until the child writes a byte into `c2p[1]`. Therefore the synchronization happens naturally through the blocking behavior of `read()`.

5. Conclusion

In this laboratory work I repeated the same basic process operations inside xv6 that were previously used on Linux. The experiments showed that the process model is conceptually the same: `fork()` creates a child, `exec()` replaces the current program, and `wait()` lets the parent collect the child's termination status.

The `pingpong` exercise demonstrated inter-process communication with pipes. Two pipes were used to create communication in both directions, and blocking `read()` calls provided synchronization between the parent and child. This work connected the high-level behavior observed in Linux with the same mechanisms inside the xv6 operating system.